

Chapter 61: Authentication

Scenario 1: The dashboard fires six refresh calls at once

Situation: Your app has a dashboard that fires five parallel API calls on load (`/profile`, `/orders`, `/notifications`, `/settings`, `/summary`). The access token expired one second before the page loaded. Network tab shows all five requests get a 401, and then — because your interceptor calls the refresh endpoint on every 401 — five separate `/auth/refresh` requests go out. The auth server logs show refresh-token reuse warnings and occasionally invalidates the session because it thinks someone is replaying a stolen refresh token.

Question: How do you design an `HttpInterceptorFn` that guarantees only one refresh call is ever in flight, and queues the other requests to retry once the new token arrives?

Answer: The core idea is a shared, memoized "refresh in progress" stream that every 401 handler subscribes to instead of independently triggering its own refresh. Use an `AuthService` with a `BehaviorSubject` /signal-backed flag and an `Observable<string>` for the in-flight refresh, built with `shareReplay(1)` so late subscribers get the same eventual token rather than starting a new HTTP call.

```
// auth-refresh.service.ts
import { Injectable, inject } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable, of, throwError } from 'rxjs';
import { catchError, finalize, shareReplay, switchMap, tap } from 'rxjs/operators';

@Injectable({ providedIn: 'root' })
export class AuthRefreshService {
  private http = inject(HttpClient);
  private accessToken = '';
  private refreshToken = '';

  // The single in-flight refresh stream. Null when no refresh is running.
  private refreshInFlight$: Observable<string> | null = null;

  getAccessToken() {
    return this.accessToken;
  }

  setTokens(access: string, refresh: string) {
    this.accessToken = access;
    this.refreshToken = refresh;
  }

  /** Called by the interceptor on 401. Safe to call concurrently. */
  refreshAccessToken(): Observable<string> {
    if (this.refreshInFlight$) {
      // Someone else already kicked off a refresh – piggyback on it.
      return this.refreshInFlight$;
    }
  }
}
```

```

this.refreshInFlight$ = this.http
  .post<{ accessToken: string; refreshToken: string }>('/auth/refresh', {
    refreshToken: this.refreshToken,
  })
  .pipe(
    tap((res) => this.setTokens(res.accessToken, res.refreshToken)),
    switchMap((res) => of(res.accessToken)),
    catchError((err) => {
      this.setTokens('', '');
      return throwError(() => err);
    }),
    finalize(() => {
      // Clear the gate ONLY after this refresh settles, so any request
      // that hits 401 before this line still gets the shared stream.
      this.refreshInFlight$ = null;
    }),
    shareReplay(1) // all concurrent subscribers replay the same result
  );

return this.refreshInFlight$;
}
}

```

```

// auth.interceptor.ts
import { HttpInterceptorFn, HttpResponseError } from '@angular/common/http';
import { inject } from '@angular/core';
import { catchError, switchMap, throwError } from 'rxjs';
import { AuthRefreshService } from '../auth-refresh.service';

export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const auth = inject(AuthRefreshService);
  const token = auth.getAccessToken();

  const authedReq = token
    ? req.clone({ setHeaders: { Authorization: `Bearer ${token}` } })
    : req;

  return next(authedReq).pipe(
    catchError((err: unknown) => {
      if (err instanceof HttpResponseError && err.status === 401 &&
!req.url.includes('/auth/refresh')) {
        return auth.refreshAccessToken().pipe(
          switchMap((newToken) =>
            next(req.clone({ setHeaders: { Authorization: `Bearer ${newToken}` } })))
        ),
        catchError((refreshErr) => {
          // Refresh itself failed – force logout downstream.
          return throwError(() => refreshErr);
        })
      )
    });
  }
  return throwError(() => err);
}

```

```

    })
  };
};

```

The key mechanics: (1) `shareReplay(1)` turns the cold refresh observable into a multicast one — subsequent `subscribe()` calls don't re-execute the HTTP POST, they just wait for the same result; (2) the `refreshInFlight$` field acts as a gate that is only cleared in `finalize`, so the window between "refresh started" and "refresh finished" is exactly when all callers share it; (3) excluding `/auth/refresh` from triggering itself prevents infinite recursion if the refresh call itself 401s. Tradeoff: this is per-tab/per-service-instance state — it does not prevent duplicate refreshes across browser tabs (see Scenario 5 for that layer using a `BroadcastChannel` or lock).

Interviewer intent: Tests whether the candidate understands RxJS multicasting (`shareReplay`) versus just slapping a boolean flag on it, and whether they see the race condition between "check flag" and "set flag" that a naive implementation misses.

Scenario 2: Users get logged out mid-form-submission

Situation: Support tickets report users typing a long form, clicking submit, and being redirected to the login page with all their work lost — even though they were "clearly still using the app." Investigation shows the access token has a 15-minute expiry and users often spend 20+ minutes filling the form before submitting, so the token is stale by the time the POST fires.

Question: How would you prevent silent data loss from expired tokens on long-running user sessions, without making tokens live "forever" (a security regression)?

Answer: Two complementary fixes: proactive refresh before expiry (rather than reactive refresh only after a 401), and preserving in-flight form state across a forced re-auth if refresh ultimately fails.

For proactive refresh, decode the JWT's `exp` claim and schedule a refresh a safety margin before expiry, using `expiresIn` returned from login rather than trusting only reactive 401 handling:

```

// token-scheduler.service.ts
import { Injectable, inject, DestroyRef } from '@angular/core';
import { timer } from 'rxjs';
import { AuthRefreshService } from '../auth-refresh.service';

@Injectable({ providedIn: 'root' })
export class TokensSchedulerService {
  private auth = inject(AuthRefreshService);
  private destroyRef = inject(DestroyRef);

  scheduleRefresh(expiresInSeconds: number) {
    // Refresh at 80% of the token lifetime, never later than 30s before expiry.
    const marginMs = Math.min(30_000, expiresInSeconds * 1000 * 0.2);
    const delay = expiresInSeconds * 1000 - marginMs;

    const sub = timer(Math.max(delay, 0)).subscribe(() => {
      this.auth.refreshAccessToken().subscribe({
        next: (newToken) => {
          // Re-decode new exp and reschedule the next proactive refresh.

```

```

    const nextExpiry = this.decodeExpiry(newToken);
    this.scheduleRefresh(nextExpiry);
  },
  error: () => {
    // Proactive refresh failed silently – don't force logout yet,
    // let the reactive 401 path on the next real request decide.
  },
});

this.destroyRef.onDestroy(() => sub.unsubscribe());
}

private decodeExpiry(token: string): number {
  const payload = JSON.parse(atob(token.split('.')[1]));
  return payload.exp - Math.floor(Date.now() / 1000);
}
}

```

For the form-loss problem specifically, add a belt-and-braces client-side guard: before submitting a long form, if the access token is within its last margin, await a refresh first (rather than letting the request fail and retry, which for POSTs with side effects can be risky to retry blindly):

```

async function submitSafely(auth: AuthRefreshService, tokenExpiringSoon: boolean, submitFn:
() => Promise<void>) {
  if (tokenExpiringSoon) {
    await firstValueFrom(auth.refreshAccessToken());
  }
  await submitFn();
}

```

Also persist form state to `sessionStorage` on an interval or on `beforeunload` / route-leave so that even if a forced logout does happen, the user doesn't lose work — restore the draft after re-login. This is standard practice for any auth flow guarding non-idempotent submissions: never let "session expired" silently discard user data; degrade to "please log in again, your draft is saved."

Interviewer intent: Tests whether the candidate defaults to purely reactive (401-driven) refresh or also designs proactive refresh, and whether they think about UX/data-loss consequences of auth expiry, not just the token mechanics.

Scenario 3: Retrying the original request after refresh replays a mutating POST twice

Situation: A junior engineer's interceptor retries the failed request after refreshing the token — but for `POST /orders`, this occasionally creates two orders when the original request had actually succeeded server-side and only the response was lost due to a flaky connection combined with the 401 handling logic being triggered from a stale cached response.

Question: What's unsafe about blindly retrying any HTTP method after a token refresh, and how do you make retries safe?

Answer: Retrying non-idempotent requests (POST, sometimes PATCH) after a network-level failure risks duplicate execution if the original request actually reached the server and was processed, but the client only *perceived* a failure (e.g., a dropped response, not a genuine 401 from an expired token check). Two mitigations:

1. **Distinguish "real" 401 (auth rejected) from transport failures.** A true 401 with a JSON body like `{ "error": "token_expired" }` is safe to retry (the server never executed business logic — it rejected at the auth-middleware layer before touching order-creation code). A 0-status or network-error is NOT safe to blindly retry for POSTs.
2. **Use idempotency keys** for all mutating requests so the server can safely dedupe retried POSTs regardless of client-side retry logic.

```
// idempotent-post.interceptor.ts
import { HttpInterceptorFn } from '@angular/common/http';

export const idempotencyInterceptor: HttpInterceptorFn = (req, next) => {
  if (req.method === 'POST' || req.method === 'PATCH') {
    const key = req.headers.get('Idempotency-Key') ?? crypto.randomUUID();
    req = req.clone({ setHeaders: { 'Idempotency-Key': key } });
  }
  return next(req);
};
```

```
// in the auth interceptor's retry branch – only retry if it's genuinely an auth rejection
function isRetryableAuthFailure(err: HttpResponse): boolean {
  return err.status === 401 && err.error?.error === 'token_expired';
}
```

The server stores the idempotency key against the resulting resource for a window (e.g., 24h); if it sees the same key again, it returns the original response instead of creating a new order. This shifts the safety guarantee from "hope the client only retries once" to "the server enforces exactly-once regardless of how many times the client retries" — the correct place to put this guarantee, since client-side retry logic can never be fully trusted (multiple tabs, flaky mobile networks, browser back-forward cache resending requests).

Interviewer intent: Tests whether the candidate understands the difference between idempotent and non-idempotent HTTP semantics in the context of retry-after-refresh logic, and whether they reach for server-side idempotency keys rather than trying to solve an inherently server-side guarantee purely on the client.

Scenario 4: Two tabs, two different users, one shared token

Situation: A user logs into Tab A as `alice@company.com`. In Tab B (same browser), they log out and log back in as `bob@company.com` for a demo. Tab A, still open, silently starts sending Bob's token on Alice's already-loaded page, and Alice's dashboard—still rendered with her data in memory—now saves changes under Bob's account because the interceptor reads the token from `localStorage` at request time.

Question: How do you architect cross-tab auth state so that logging in/out in one tab doesn't corrupt the session of another tab that's mid-use with different assumptions?

Answer: The fundamental bug is storing the *only* copy of identity in a shared, tab-agnostic store (`localStorage`) with no invalidation signal to already-open tabs. Two options depending on desired behavior:

Option A — synchronize all tabs to a single session (most common for consumer apps): use a `storage` event listener (or `BroadcastChannel`) to detect token changes made by other tabs and force a hard reload/reinitialize when the user identity changes, rather than let the tab keep running with stale in-memory state pointed at new credentials.

```
// cross-tab-auth.service.ts
import { Injectable, NgZone, inject } from '@angular/core';
import { Router } from '@angular/router';

@Injectable({ providedIn: 'root' })
export class CrossTabAuthService {
  private channel = new BroadcastChannel('auth');
  private router = inject(Router);
  private zone = inject(NgZone);
  private currentUserId: string | null = null;

  init(currentUserId: string) {
    this.currentUserId = currentUserId;

    this.channel.onmessage = (msg) => {
      this.zone.run(() => {
        if (msg.data.type === 'LOGOUT' || (msg.data.type === 'LOGIN' && msg.data.userId !==
this.currentUserId)) {
          // Identity changed in another tab - this tab's in-memory state is
          // no longer trustworthy. Force a clean reload rather than patch state.
          window.location.href = '/login?reason=session-changed';
        }
      });
    };
  }

  broadcastLogin(userId: string) {
    this.channel.postMessage({ type: 'LOGIN', userId });
  }

  broadcastLogout() {
    this.channel.postMessage({ type: 'LOGOUT' });
  }
}
```

Option B — genuinely independent per-tab sessions: move tokens out of `localStorage` (shared across tabs) into `sessionStorage` or an in-memory service, each tab logging in independently, at the cost of losing "log in once, all tabs are authenticated" convenience and needing the refresh token in an `HttpOnly` cookie scoped correctly, or a BFF pattern. This is the right choice for admin tools/kiosks where multi-identity-per-browser is a deliberate feature, not a bug.

Most production apps choose Option A because it matches user expectations (a bank app should not let you be silently logged in as two people in two tabs) — the fix is to treat identity as a broadcast fact, not a per-tab cache, and hard-reload on mismatch rather than trying to reconcile in-flight component state, which is fragile and easy to get wrong (stale closures, cached signals, in-memory NgRx store still holding Alice's data).

Interviewer intent: Tests awareness that `localStorage` is shared mutable global state across tabs and that reactive frameworks need an explicit cross-tab synchronization primitive (`BroadcastChannel` / `storage` event) — a scenario many candidates have never had to solve in practice.

Scenario 5: Logging out in one tab leaves other tabs "logged in" until their next API call

Situation: User clicks "Log out" in Tab A. Tab B, showing a dashboard with auto-refreshing widgets every 30 seconds, keeps displaying data and even lets the user click buttons for up to 30 seconds after logout, because it hasn't made a network call yet to discover the token is gone.

Question: Design a logout mechanism that propagates instantly across all open tabs, not just eventually via a failed API call.

Answer: Use `BroadcastChannel` (or the `storage` event as a fallback for older environments) to push an explicit, immediate logout signal, and have every tab's root injector subscribe to it and react synchronously — clearing in-memory tokens, redirecting, and disabling further UI interaction — rather than waiting for an HTTP round trip to fail.

```
// logout-broadcast.service.ts
import { Injectable, inject } from '@angular/core';
import { Router } from '@angular/router';
import { AuthRefreshService } from '../auth-refresh.service';

@Injectable({ providedIn: 'root' })
export class LogoutBroadcastService {
  private channel = new BroadcastChannel('auth-logout');
  private router = inject(Router);
  private auth = inject(AuthRefreshService);

  listen() {
    this.channel.onmessage = (event) => {
      if (event.data === 'LOGOUT') {
        this.hardLogoutLocalStorage();
      }
    };
  }

  logoutEverywhere() {
    this.hardLogoutLocalStorage();
    this.channel.postMessage('LOGOUT');
  }

  private hardLogoutLocalStorage() {
    this.auth.setTokens('', '');
    localStorage.removeItem('refreshToken');
  }
}
```

```
    this.router.navigateByUrl('/login');
  }
}
```

Bootstrap this once at app start (e.g., in `app.config.ts` via an `APP_INITIALIZER`-equivalent `provideAppInitializer` in modern Angular, or simply calling `.listen()` in the root component's constructor):

```
// app.component.ts
export class AppComponent {
  private logoutBroadcast = inject(LogoutBroadcastService);
  constructor() {
    this.logoutBroadcast.listen();
  }
}
```

Additionally, revoke the refresh token server-side on logout (not just client-side token deletion) — otherwise a stolen refresh token captured before logout remains valid indefinitely. The server should maintain a refresh-token blocklist/rotation table and invalidate the specific token (or entire session family) on logout. Client-side-only "logout" that merely deletes `localStorage` gives a false sense of security if the token was already exfiltrated (e.g., via XSS) before logout was clicked.

Interviewer intent: Tests whether the candidate treats logout as purely a client-side state change or recognizes it needs both instant cross-tab propagation AND server-side token revocation — many candidates only solve half of this.

Scenario 6: Refresh token rotation causes users to get logged out in a loop

Situation: After implementing refresh token rotation (each refresh call returns a *new* refresh token and invalidates the old one, standard OWASP guidance to detect token theft), users started reporting being logged out constantly, sometimes mid-session. Server logs show "refresh token reuse detected — revoking session family" firing even for legitimate single-user traffic.

Question: Why does rotation cause this, and how do you fix client-side handling so legitimate use doesn't trigger the theft-detection false positive?

Answer: This is almost always a race condition: two requests hit 401 near-simultaneously, both read the *same* (now-stale) refresh token from storage, and both call `/auth/refresh`. The server issues token R2 to the first caller and immediately invalidates R1. The second caller's request — already in flight with R1 — arrives moments later, and the server sees "R1 used again after rotation" and interprets it as replay/theft, revoking the entire token family (this is by design for security, but the client caused it by refreshing twice concurrently).

The fix is exactly the single-flight refresh gate from Scenario 1 (`shareReplay(1)` around one shared refresh observable) — it must be airtight so that under no circumstance do two refresh calls fire from the same client using the same stale refresh token. Additionally, harden against cross-tab duplication (Scenario 4/5 concerns) since rotation makes cross-tab races catastrophic — if Tab A and Tab B both hold R1 and both try to refresh independently, one of them will always trigger the reuse-detection revocation.

```
// Cross-tab-safe refresh: use a Web Lock so only one tab-refreshes at a time,
// even across separate JS execution contexts (not just within one tab's RxJS pipe).
async function refreshWithCrossTabLock(auth: AuthRefreshService): Promise<string> {
  return navigator.locks.request('refresh-token-lock', async () => {
    // Re-read the token AFTER acquiring the lock – another tab may have
    // already refreshed while we were waiting for the lock.
    const currentToken = localStorage.getItem('refreshToken');
    const cachedFreshToken = sessionStorage.getItem('accessTokenIssuedForRefresh_' +
currentToken);
    if (cachedFreshToken) {
      return cachedFreshToken; // another tab already rotated for us
    }
    const result = await firstValueFrom(auth.refreshAccessToken());
    return result;
  });
}
```

The `Web Locks API` (`navigator.locks`) is the correct primitive here because it coordinates across tabs/workers within the same origin, unlike `RxJS shareReplay` which only coordinates within one JS heap (one tab). Combine both: `shareReplay` for intra-tab de-duplication, `navigator.locks` for inter-tab de-duplication. Also recommend a small grace window server-side (allow the immediately-prior refresh token to succeed once within e.g. 2 seconds of rotation, to tolerate clock/network skew) — a pure zero-tolerance policy is technically more secure but operationally fragile against exactly this kind of legitimate race.

Interviewer intent: Tests deep understanding of refresh token rotation's security model (reuse detection) and whether the candidate can connect a backend security feature to a client-side concurrency bug — a favorite "senior-level" trick question because the fix spans both layers.

Scenario 7: Migrating from session cookies to JWT breaks "stay logged in for 30 days"

Situation: The app used server-side sessions with an `HttpOnly` cookie that lasted 30 days with sliding expiration (any request extended it). After migrating to JWT (short-lived access token + refresh token), product complains that "remember me" no longer works the same way — some users are fully logged out after a week of daily use, which never happened before.

Question: What's different about session semantics vs JWT semantics that explains this regression, and how do you replicate "sliding 30-day session" behavior with JWTs?

Answer: Server-side sessions are inherently stateful and mutable — "extend expiry" is just an UPDATE on a row keyed by session ID, and the cookie itself never needs to change. JWTs are stateless and immutable by design: the token's own `exp` claim is baked in at issuance and cannot be "extended" without issuing a brand-new token. If your refresh token has a fixed 30-day absolute expiry set at login and is never rotated with a renewed expiry on each use, then day 31 always logs the user out — regardless of activity — unlike the old sliding-session cookie.

The fix is to implement sliding expiration explicitly for the refresh token: each time the refresh token is used, issue a new refresh token with a renewed 30-day expiry (this is also naturally compatible with rotation from Scenario 6 — you get sliding expiration "for free" as a side effect of rotating on every use, as long as the *new* token's expiry is reset to `now + 30d`, not inherited from the original).

```
// Server-side conceptual sketch – not Angular code, but relevant context the interviewer expects you to know:
// POST /auth/refresh
// 1. validate incoming refresh token (signature + not expired + not revoked)
// 2. revoke incoming refresh token (rotation)
// 3. issue NEW refresh token with exp = now + 30 days (sliding window)
// 4. issue NEW access token with exp = now + 15 minutes
```

On the Angular side, this requires the app to actually exercise the refresh flow periodically even for "idle-looking" sessions, so the 30-day clock keeps sliding as long as the user opens the app at least once every 30 days — this is naturally satisfied by the proactive-refresh scheduler from Scenario 2, since every access-token refresh also rotates and renews the refresh token. The key conceptual point to raise in an interview: **JWTs need every "session-like" behavior (sliding expiry, instant revocation, server-side logout) to be deliberately re-engineered**, because you're trading built-in stateful session semantics for stateless scalability — there's no free lunch, and a lift-and-shift migration that assumes JWTs "just work like cookies" will regress exactly this kind of behavior.

Interviewer intent: Tests conceptual understanding of the stateful-session vs stateless-JWT tradeoff beyond rote "JWT vs cookie" trivia — specifically whether the candidate can reason about *why* a migration silently changes session-length semantics.

Scenario 8: Migrating from session cookies to JWT — CSRF disappears but XSS risk goes up

Situation: During the same cookie-to-JWT migration, a security reviewer flags that the team removed CSRF token handling entirely (reasonable, since JWTs sent via `Authorization` header aren't auto-attached like cookies, so CSRF doesn't apply the same way) — but flags that the team is now storing the JWT in `localStorage`, which the reviewer says is worse.

Question: Explain the security tradeoff being made here and how you'd store tokens to get the best of both worlds.

Answer: The reviewer is right to flag it. Cookies (especially `HttpOnly`, `Secure`, `SameSite=Strict`) are immune to JavaScript theft via XSS because client-side JS cannot read them at all — their weakness is CSRF, since browsers auto-attach cookies to every request to that domain, including ones triggered by a malicious third-party site. JWTs in `localStorage` invert the tradeoff: immune to CSRF (nothing is auto-attached; the attacker's page can't read your JWT to forge a header), but any XSS vulnerability anywhere in the app (a single unsanitized `innerHTML`, a compromised third-party script) lets an attacker read `localStorage` directly and exfiltrate the token wholesale — which is arguably worse, since a stolen JWT is fully portable and usable outside the browser (e.g., replayed from `curl`), whereas a stolen `HttpOnly` cookie can only be *used* by an attacker running code in the victim's browser context (still bad, but a narrower blast radius for some attack shapes) — and no CSRF-style forgeries are needed either way once JS execution is compromised.

The best-of-both-worlds pattern recommended by OWASP and used by most mature Angular+API stacks is the **BFF (Backend-For-Frontend) pattern**: keep the refresh token in an `HttpOnly`, `Secure`, `SameSite=Strict` cookie (never touchable by JS, so immune to XSS-based exfiltration), and keep only the short-lived access token in memory (a plain TypeScript variable in a singleton service, never `localStorage` / `sessionStorage`) so a page reload naturally clears it and an XSS payload would need to actively hook into your running app's memory rather than just reading storage.

```
// in-memory-only access token store – no persistence, cleared on reload by design
import { Injectable } from '@angular/core';

@Injectable({ providedIn: 'root' })
export class InMemoryTokenStore {
  private accessToken = ''; // intentionally NOT persisted anywhere

  get() { return this.accessToken; }
  set(token: string) { this.accessToken = token; }
  clear() { this.accessToken = ''; }
}
```

```
// bootstrap.ts – on every app load, silently re-establish the access token
// using the HttpOnly refresh cookie (sent automatically, invisible to JS)
export function initializeAuth(http: HttpClient, store: InMemoryTokenStore) {
  return () => firstValueFrom(
    http.post<{ accessToken: string }>('/auth/refresh', {}, { withCredentials: true })
      .pipe(
        tap((res) => store.set(res.accessToken)),
        catchError(() => of(null)) // not logged in yet – fine, guards handle it
      )
  );
}
```

Since the refresh cookie is `SameSite=Strict/Lax` and `HttpOnly`, CSRF risk on the refresh endpoint is minimal (and can be further hardened with a double-submit CSRF token on state-changing endpoints if `SameSite=Lax` is required for cross-site redirect flows like OAuth). This pattern means an XSS vulnerability can still cause damage (steal the current in-memory access token and use it for its short remaining lifetime), but caps the damage window to the access token's TTL (e.g., 15 minutes) instead of granting the attacker a durable, replayable 30-day credential.

Interviewer intent: Tests whether the candidate can articulate the CSRF-vs-XSS tradeoff precisely (not just "cookies bad, localStorage bad") and knows the BFF/hybrid storage pattern as the actual industry-recommended mitigation, not just a list of pros/cons with no resolution.

Scenario 9: OAuth2/OIDC redirect flow drops query params after login

Situation: A user deep-links to `/reports/quarterly?year=2026®ion=EU` while logged out. Your route guard redirects to the OIDC provider's login page. After successful login, the OIDC provider redirects back to your app's callback URL — but the user lands on the app's plain dashboard, having lost the original destination and query params entirely.

Question: How do you preserve the originally requested URL (including query params) through a full OIDC redirect round-trip, and what security pitfalls must you avoid while doing so?

Answer: The redirect must carry the "return-to" URL through the OIDC `state` parameter (the mechanism specifically designed for this, and also required for CSRF protection of the auth flow itself), not through your own ad-hoc query param or `localStorage`, because the user's browser fully navigates away to the identity provider and back — any purely client-side JS state (component fields, services) is destroyed, and even `sessionStorage` can be unreliable if the provider redirect involves a different browsing context in some edge cases (though `sessionStorage` usually does survive a same-tab redirect and is a reasonable fallback).

```
// auth.guard.ts – functional guard capturing intended destination
import { CanActivateFn } from '@angular/router';
import { inject } from '@angular/core';
import { AuthService } from './auth.service';

export const authGuard: CanActivateFn = (route, state) => {
  const auth = inject(AuthService);

  if (auth.isAuthenticated()) {
    return true;
  }

  // state.url includes the full path + query string the user actually requested
  auth.redirectToLogin(state.url);
  return false;
};
```

```
// auth.service.ts – encode the return URL into the OIDC `state` param
redirectToLogin(returnUrl: string) {
  const state = btoa(JSON.stringify({ returnUrl, nonce: crypto.randomUUID() }));
  // Persist the nonce so we can verify the returned `state` wasn't tampered with –
  // this is what makes `state` a CSRF defense, not just a data-passing convenience.
  sessionStorage.setItem('oidc_state_nonce', state);

  const authUrl = new URL('https://idp.example.com/authorize');
  authUrl.searchParams.set('client_id', this.clientId);
  authUrl.searchParams.set('redirect_uri', this.callbackUrl);
  authUrl.searchParams.set('response_type', 'code');
  authUrl.searchParams.set('scope', 'openid profile email');
  authUrl.searchParams.set('state', state);
  window.location.href = authUrl.toString();
}
```

```
// callback.component.ts – verify state, then restore the original destination
export class CallbackComponent {
  private router = inject(Router);
  private route = inject(ActivatedRoute);
  private auth = inject(AuthService);

  constructor() {
    const returnedState = this.route.snapshot.queryParamMap.get('state');
    const code = this.route.snapshot.queryParamMap.get('code');
    const storedState = sessionStorage.getItem('oidc_state_nonce');
```

```

    if (!returnedState || returnedState !== storedState) {
      // state mismatch – possible CSRF/injection attempt on the auth flow. Abort.
      this.router.navigateByUrl('/login?error=state_mismatch');
      return;
    }

    sessionStorage.removeItem('oidc_state_nonce');
    const { returnUrl } = JSON.parse(atob(returnedState));

    this.auth.exchangeCodeForTokens(code!).subscribe(() => {
      this.router.navigateByUrl(returnUrl || '/dashboard');
    });
  }
}

```

Security pitfalls to call out explicitly: (1) never trust `returnUrl` blindly — validate it's a relative, same-origin path (not `https://evil.com`) before navigating, to prevent open-redirect attacks where a crafted deep link tricks the app into bouncing an authenticated user to an attacker site post-login; (2) the `state` param must be unpredictable/unique per auth attempt and verified on return — this is what stops an attacker from initiating their own OIDC flow and tricking a victim into completing it (login CSRF); (3) also validate/verify the OIDC `nonce` claim inside the returned ID token if using implicit/hybrid flow, a separate mechanism from `state`, to prevent ID token replay.

Interviewer intent: Tests whether the candidate actually understands the purpose of the OIDC `state` parameter (CSRF protection + data carrier) versus treating it as a black box, and whether they think about open-redirect validation, a commonly missed detail.

Scenario 10: Silent SSO check causes a flash of login page on every hard refresh

Situation: The app integrates SSO via OIDC "silent renew" (a hidden iframe hitting the IdP's `/authorize?prompt=none`). On every hard page refresh, users briefly see the login page flash for ~300-800ms before the silent check completes and they're redirected back into the app — jarring and looks broken, especially on slower connections.

Question: How do you redesign the bootstrap sequence so the app doesn't render (or render the wrong UI) before the silent-SSO check resolves?

Answer: The root cause is treating auth-check as "just another async thing that happens after the app renders," so the router already resolves the default/login route and paints it before the silent iframe check finishes. The fix is to block app rendering (or at minimum, block routing) on an app initializer that resolves the auth state definitively before the Angular router makes its first navigation decision.

```

// app.config.ts
import { ApplicationConfig, provideAppInitializer, inject } from '@angular/core';
import { AuthService } from './auth.service';
import { firstValueFrom } from 'rxjs';

export const appConfig: ApplicationConfig = {
  providers: [

```

```

// ... other providers
provideAppInitializer(() => {
  const auth = inject(AuthService);
  // This MUST resolve (success or failure) before bootstrapping continues,
  // so the router's first navigation already knows the real auth state.
  return firstValueFrom(auth.trySilentSsoCheck()).catch(() => null);
}),
],
};

```

```

// auth.service.ts
trySilentSsoCheck(): Observable<boolean> {
  return new Observable<boolean>((subscriber) => {
    const iframe = document.createElement('iframe');
    iframe.style.display = 'none';
    iframe.src = this.buildSilentAuthorizeUrl(); // prompt=none,
    response_mode=fragment/query

    const timeout = setTimeout(() => {
      cleanup();
      subscriber.next(false); // IdP took too long or user isn't SSO-authenticated
      subscriber.complete();
    }, 4000);

    const messageHandler = (event: MessageEvent) => {
      if (event.origin !== this.idpOrigin) return;
      if (event.data?.type === 'silent-auth-success') {
        this.setTokens(event.data.accessToken, event.data.refreshToken);
        cleanup();
        subscriber.next(true);
        subscriber.complete();
      } else if (event.data?.type === 'silent-auth-failure') {
        cleanup();
        subscriber.next(false);
        subscriber.complete();
      }
    };

    function cleanup() {
      clearTimeout(timeout);
      window.removeEventListener('message', messageHandler);
      iframe.remove();
    }

    window.addEventListener('message', messageHandler);
    document.body.appendChild(iframe);
  });
}

```

Additionally, show a lightweight, branded splash/loading screen (via `index.html` static markup, shown before Angular even bootstraps, removed once bootstrap completes) rather than a blank white screen during this initializer — this avoids both the "flash of login page" AND a flash of blank white, giving a clean perceived-loading experience. Set a hard timeout (e.g., 3-4 seconds) on the silent check so a slow/unreachable IdP doesn't hang app bootstrap indefinitely — fail open to the normal login page after the timeout rather than leaving users stuck on a splash screen forever.

Interviewer intent: Tests understanding of Angular's app initialization lifecycle (`provideAppInitializer`) and whether the candidate knows to gate routing decisions on async auth resolution rather than letting the router race ahead of an SSO check — a very common real-world SSO integration bug.

Scenario 11: A guard reads a signal synchronously and always sees "logged out" right after login

Situation: Right after a successful login, the app calls `router.navigateByUrl('/dashboard')`, but the `authGuard` on that route blocks it, redirecting straight back to `/login`, creating a redirect loop. Removing the guard "fixes" it, confirming the guard is the culprit. The login service does set the auth signal synchronously right before navigating.

Question: Debug why a synchronously-set signal isn't visible to the guard evaluating immediately afterward, and fix it.

Answer: This is rarely actually an "async timing" issue with signals themselves (signals ARE synchronous and immediately consistent) — the far more common real cause is that login sets state on one instance of the service, but the guard injects a *different* instance, because the service is provided in a lazy-loaded feature module/route's own injector (or `providedIn: 'component' / route-level providers array`) instead of the root injector, so `inject(AuthService)` in the guard and in the login component resolve to two separate instances with two separate signals.

```
// BUG: providedIn root is fine here, but if AuthService is ALSO listed in a
// lazy-loaded route's `providers` array, Angular creates a child-injector-scoped
// instance for anything resolved within that route subtree, shadowing the root one.

// routes.ts – the bug
export const routes: Routes = [
  {
    path: 'dashboard',
    canActivate: [authGuard],
    providers: [AuthService], // <-- creates a SECOND instance scoped to this route
    loadComponent: () => import('./dashboard.component').then(m => m.DashboardComponent),
  },
];
```

```
// FIX: rely solely on the root-provided singleton; never re-list it in route providers
@Injectable({ providedIn: 'root' })
export class AuthService {
  private _isLoggedIn = signal(false);
  readonly isLoggedIn = this._isLoggedIn.asReadonly();
```

```
login(credentials: Credentials) {
  return this.http.post('/auth/login', credentials).pipe(
    tap(() => this._isLoggedIn.set(true)) // synchronous, immediately visible everywhere
  );
}
}

// routes.ts – no duplicate provider
export const routes: Routes = [
  {
    path: 'dashboard',
    canActivate: [authGuard],
    loadComponent: () => import('./dashboard.component').then(m => m.DashboardComponent),
  },
];
```

```
// auth.guard.ts
export const authGuard: CanActivateFn = () => {
  const auth = inject(AuthService); // resolves the SAME root instance as the login flow
  const router = inject(Router);
  if (auth.isLoggedIn()) return true;
  router.navigateByUrl('/login');
  return false;
};
```

A second, less common but real variant of this bug: the login `tap()` sets the signal, but `navigateByUrl` is called from *inside* the `subscribe()` callback while the HTTP response `tap` and the navigation are racing against a `combineLatest/withLatestFrom` elsewhere that captured a snapshot of the signal *before* the update (rare with signals since they don't have this staleness problem the way BehaviorSubjects piped through stale closures do, but worth mentioning: always read `signal()` at guard-evaluation time, never cache its value in a captured variable at construction time). The universal fix and lesson: services holding auth state must be true app-wide singletons — `providedIn: 'root'` and never re-declared in any route/component `providers` array — otherwise Angular's hierarchical DI silently gives different consumers different instances.

Interviewer intent: Tests deep knowledge of Angular's hierarchical dependency injection and whether the candidate can diagnose a "state isn't shared" bug as a DI scoping issue rather than assuming it's a signals/async problem — a classic "looks like reactivity bug, is actually DI bug" trap.

Scenario 12: Access token still valid per `exp`, but API rejects it as "expired" — clock skew

Situation: A subset of users (traced to a specific corporate network with an incorrectly configured NTP server) get logged out within seconds of logging in. Decoding their JWT client-side shows `exp` is a full 10 minutes in the future. The server insists the token is expired. Support can't reproduce it on any correctly-configured machine.

Question: What's causing the discrepancy, and how do you make the client resilient to it without weakening actual security?

Answer: JWT expiry validation is inherently clock-based, and if either the client's system clock (used when the client itself decides whether to preemptively refresh) or — far more critically — the *server's* clock is skewed relative to real time, `exp` comparisons can disagree. In this case, it's most likely the client machine's clock is skewed forward (thinks it's later than it really is), causing the client to *display* an `exp` that looks fine locally, while the actual issued timestamp baked into the token (`iat`) was computed against the *server's* correct clock — so the token might genuinely have a legitimately short remaining lifetime by server-truth-time, or in the reverse case, the server's own clock could be skewed and rejecting genuinely-valid tokens.

This is not really an Angular-specific bug to "fix" client-side beyond defensive handling — the correct owners of the fix are: (1) ensure the auth server has correct NTP sync (this is the real root cause almost always, since well-run auth servers are the source of truth and should never be skewed); (2) apply a small clock-skew tolerance (`leeway`, e.g., 30-60 seconds) in the server's JWT validation library, which virtually every JWT library supports (e.g., `jsonwebtoken`'s `clockTolerance` option) — this is standard, safe practice and does not meaningfully weaken security, since it only affects boundary-second rejections, not the actual expiry window.

On the Angular side, the resilient pattern is to **never trust the client's own decoded `exp` as the sole authority for "is my token still good"** — always let the server be the arbiter via the actual 401 response, and only use client-side `exp` decoding as a *heuristic* for proactive refresh scheduling (Scenario 2), never as a hard gate that blocks a request from being sent at all:

```
// ANTI-PATTERN: don't do this - client clock skew makes this actively wrong
function isValidLocally(token: string): boolean {
  const { exp } = JSON.parse(atob(token.split('.')[1]));
  return exp * 1000 > Date.now(); // Date.now() is only as trustworthy as this device's
  clock
}
```

```
// BETTER: always attempt the real request; let the server's 401 be authoritative;
// use local exp decode only to opportunistically pre-refresh, never to block sending.
```

```
// If you must guard against sending obviously-dead tokens (e.g., to save a
// wasted round trip), use a generous tolerance rather than an exact comparison:
function looksExpiredWithTolerance(exp: number, toleranceSeconds = 120): boolean {
  return exp * 1000 < Date.now() - toleranceSeconds * 1000; // only skip if WAY past expiry
}
```

Also log the skew when detected (compare the server's `Date` response header against `Date.now()` on any API response) so ops can proactively flag misconfigured client environments, and consider surfacing a user-facing warning ("Your device's clock appears to be incorrect, which may cause login issues") since this is genuinely a client-environment problem outside the app's control.

Interviewer intent: Tests whether the candidate understands that JWT expiry is fundamentally a distributed-systems clock-trust problem, not purely a token-format problem, and knows the standard mitigation (`clockTolerance` / `leeway`) rather than trying to "fix" it entirely in Angular code.

Scenario 13: Guard blocks navigation because auth state hasn't loaded yet on hard refresh

Situation: On a hard refresh of `/dashboard` (not a client-side navigation), the `authGuard` runs before the app has had a chance to restore the session from the refresh-token cookie/silent SSO check, sees `isLoggedIn() === false`, and redirects to `/login` — even though the user has a perfectly valid session that just hadn't finished loading yet.

Question: How do you make a functional guard properly wait for an async "is the user actually authenticated" determination instead of racing ahead on an uninitialized default value?

Answer: The guard must return an `Observable<boolean> / Promise<boolean>` that Angular's router natively awaits, rather than synchronously reading a signal that hasn't been populated yet. Combine this with the `provideAppInitializer` approach from Scenario 10 as defense-in-depth, but the guard itself should also be independently robust in case it's ever hit before that initializer fully completes (e.g., lazy-loaded route resolved mid-initialization in edge cases, or simply as a second layer of safety).

```
// auth.service.ts
@Injectable({ providedIn: 'root' })
export class AuthService {
  private authState$ = new BehaviorSubject<'unknown' | 'authenticated' | 'unauthenticated'>('unknown');

  /** Emits exactly once the auth check has genuinely resolved, filtering out 'unknown'. */
  readonly resolvedAuthState$ = this.authState$.pipe(
    filter((state) => state !== 'unknown'),
    take(1)
  );

  markAuthenticated() { this.authState$.next('authenticated'); }
  markUnauthenticated() { this.authState$.next('unauthenticated'); }
}
```

```
// auth.guard.ts
import { CanActivateFn } from '@angular/router';
import { inject } from '@angular/core';
import { map } from 'rxjs/operators';
import { AuthService } from './auth.service';
import { Router } from '@angular/router';

export const authGuard: CanActivateFn = () => {
  const auth = inject(AuthService);
  const router = inject(Router);

  // The router will subscribe and wait for this observable to emit before
  // deciding navigation – it does NOT evaluate synchronously against a
  // possibly-uninitialized value.
  return auth.resolvedAuthState$.pipe(
    map((state) => {
      if (state === 'authenticated') return true;
      return router.parseUrl('/login'); // returning a UrlTree redirects cleanly
    })
  );
}
```

```
    })
  );
};
```

The critical design point: `authState$` starts as `'unknown'`, not `false / true` — this three-state model (unknown / authenticated / unauthenticated) is what prevents the guard from ever treating "haven't checked yet" the same as "checked and confirmed logged out." A common bug is modeling this as a plain boolean defaulting to `false`, which makes "not yet known" indistinguishable from "definitely not logged in," causing exactly this race. The `resolvedAuthState$` observable, filtered to skip `'unknown'` and take only the first real resolution, gives the router something it can safely await.

Interviewer intent: Tests whether the candidate knows guards can return observables/promises (not just booleans) and, more importantly, whether they design auth state as a three-state model (unknown/yes/no) rather than a naive boolean — a subtle but very common production bug.

Scenario 14: Refresh succeeds but the retried request still uses the old token

Situation: After implementing the refresh-and-retry interceptor pattern, QA reports that occasionally the retried request after a successful refresh *still* comes back 401 — but manually checking, the new token is clearly valid and present in storage. Logs show the retried request's `Authorization` header contains the OLD token.

Question: Diagnose why the retried request uses the stale token even though the refresh visibly succeeded before the retry fires.

Answer: This is a classic closure-capture bug: the retried request is built by cloning the *original* `req` object (correct), but the token attached to the clone header is read from a variable captured at the time the request pipeline started, not re-read fresh after the refresh resolves.

```
// BUG: `token` was captured once at the top of the function, before refresh happened
export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const auth = inject(AuthRefreshService);
  const token = auth.getAccessToken(); // captured HERE, before any refresh occurs

  const authedReq = req.clone({ setHeaders: { Authorization: `Bearer ${token}` } });

  return next(authedReq).pipe(
    catchError((err) => {
      if (err.status === 401) {
        return auth.refreshAccessToken().pipe(
          switchMap(() => {
            // BUG: reusing the stale `token` closure variable instead of the new one!
            next(req.clone({ setHeaders: { Authorization: `Bearer ${token}` } })))
          )
        );
      }
    })
  );
  return throwError(() => err);
}
```

```
};
```

```
// FIX: use the token EMITTED by refreshAccessToken(), not the outer closure variable
export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const auth = inject(AuthRefreshService);
  const token = auth.getAccessToken();

  const authedReq = token
    ? req.clone({ setHeaders: { Authorization: `Bearer ${token}` } })
    : req;

  return next(authedReq).pipe(
    catchError((err) => {
      if (err instanceof HttpResponse && err.status === 401) {
        return auth.refreshAccessToken().pipe(
          switchMap((freshToken) =>
            // Use freshToken from the refresh result directly – never the stale closure.
            next(req.clone({ setHeaders: { Authorization: `Bearer ${freshToken}` } })))
          );
      }
      return throwError(() => err);
    })
  );
};
```

The general lesson for interceptor code (and RxJS pipelines generally): once you're inside a `switchMap` / `mergeMap` callback chained after an async operation that mutates shared state, always consume the *emitted value* from that operation for anything time-sensitive, rather than reaching back out to a variable captured earlier in the same function's scope — that outer variable reflects the world as it was before the async operation ran, and RxJS gives you no automatic re-evaluation of already-captured `const`s. This bug is especially sneaky because it only manifests intermittently — specifically whenever a refresh actually had to happen — making it easy to miss in code review of otherwise-correct-looking interceptor code.

Interviewer intent: Tests careful code-reading ability and understanding of JavaScript closures interacting with RxJS operators — a bug that looks like an auth/token bug but is really a fundamental "stale closure variable" bug, testing whether the candidate can spot it by reading actual code rather than reasoning abstractly.

Scenario 15: Refresh token stored in `localStorage` gets silently wiped by Safari's ITP

Situation: iOS Safari users report being logged out "randomly," roughly correlating with periods of exactly 7 days of inactivity, but never on Android/Chrome. No server-side session expiry matches 7 days.

Question: What Safari-specific browser behavior explains this, and how do you architect storage to be resilient to it?

Answer: Safari's Intelligent Tracking Prevention (ITP) purges *all* script-writable storage (`localStorage`, `IndexedDB`, cookies not set with special exemptions) for a site after 7 days of no user interaction with that site as a top-level, user-initiated navigation — a deliberate anti-tracking measure, not a bug, but one that silently destroys long-lived refresh tokens stored in `localStorage` for any user who doesn't visit the site directly (as opposed to via an embedded iframe/redirect) at least once a week.

The architecturally correct fix is exactly the BFF/cookie pattern from Scenario 8: store the refresh token in an `HttpOnly`, `Secure` cookie set by the *server* (`Set-Cookie` response header), not in `localStorage`. Server-set `HttpOnly` cookies are explicitly exempted from ITP's 7-day script-writable-storage purge — ITP specifically targets storage that JavaScript can read/write (used for cross-site tracking), and `HttpOnly` cookies are by definition invisible to JavaScript, so they don't fall under this policy the same way (though ITP does apply separate, generally longer-lived caps to cookies in some configurations — always verify current Safari behavior, as ITP rules evolve, but the qualitative point that `HttpOnly` cookies are far more durable than `localStorage` under ITP holds).

```
// Client-side: never persist the refresh token yourself; trust the browser's
// cookie jar (populated via Set-Cookie from the login/refresh response) and
// always send credentials so the HttpOnly cookie is included automatically.
export function loginRequest(http: HttpClient, credentials: Credentials) {
  return http.post('/auth/login', credentials, { withCredentials: true });
  // Server responds with:
  //   Set-Cookie: refreshToken=...; HttpOnly; Secure; SameSite=Lax; Path=/auth
  // The browser stores and re-sends this automatically; Angular code never
  // touches the refresh token string directly.
}
```

```
// HttpClient global config must opt in to sending cookies cross-origin if
// the API is on a different subdomain than the app.
import { provideHttpClient, withInterceptors, withXsrfConfiguration } from
  '@angular/common/http';

export const appConfig: ApplicationConfig = {
  providers: [
    provideHttpClient(
      withInterceptors([authInterceptor]),
      withXsrfConfiguration({ cookieName: 'XSRF-TOKEN', headerName: 'X-XSRF-TOKEN' })
    ),
  ],
};
```

As a secondary mitigation for genuinely long-dormant users (>7 days, even with cookies), design the UX to gracefully handle "your session fully expired, please log in again" rather than treating it as a bug — some level of forced re-auth after long dormancy is reasonable and expected; the actual bug being fixed here is *unexpectedly* short (1-week) logouts for users who believed themselves "remembered for 30 days," which the cookie-based fix genuinely resolves.

Interviewer intent: Tests real-world, platform-specific knowledge (Safari ITP) that only surfaces from actually shipping and supporting a production app — distinguishes candidates with hands-on production auth experience from those who've only worked with auth in a tutorial/greenfield context.

Scenario 16: SSO users can log into the app but get 403 on every API call

Situation: After integrating enterprise SSO (SAML via an OIDC bridge), users successfully authenticate and the app shows their name/avatar correctly — but every single API call returns 403 Forbidden. Non-SSO users (regular email/password) work fine. The JWT for SSO users decodes fine and looks structurally identical to a normal user's token.

Question: Walk through how you'd isolate whether this is a client-side, gateway, or backend authorization issue, and what's the most likely root cause given SSO is specifically implicated.

Answer: Since the token decodes fine and the *authentication* layer (proving who the user is) clearly works (name/avatar render correctly, meaning the app parsed valid claims), the bug is almost certainly in *authorization* (deciding what the authenticated user is allowed to do), and SSO being uniquely implicated strongly suggests a claims-mapping problem: SSO-provisioned users likely have a different claims shape than locally-registered users — e.g., missing a `roles` or `permissions` claim entirely, or the IdP names it differently (`groups` vs `roles`, or nested under a namespaced custom claim like `https://yourapp.com/roles` per OIDC's recommendation for custom claims), and the API's authorization middleware silently treats "claim absent" as "no permissions" rather than throwing a clear error.

Isolation steps, in order (this is the process the interviewer wants to hear, not just the final answer):

1. **Decode the actual JWT payload for an affected SSO user** (not just check it "looks fine" structurally) and diff it field-by-field against a working non-SSO user's token — specifically compare `roles` / `permissions` / `groups` / `aud` / `scope` claims.
2. **Check the `aud` (audience) claim** — a common SSO misconfiguration issues tokens scoped to the identity provider's own client ID rather than your API's expected audience, causing the API's JWT validation middleware to accept the signature but reject on audience mismatch (which some middleware surfaces as 403 rather than 401, muddying the signal).
3. **Check for a role-mapping step that was never configured for the SSO path** — many apps have a "sync IdP groups to internal roles" step (e.g., an IdP group called `Engineering-Team` needs to map to an internal `role: editor`) that's often a manual config entry per IdP integration and is easy to forget during initial SSO rollout.

```
// A debugging utility component/service (dev-only) to make this diagnosis fast in the app itself:
export function debugDecodeJwtClaims(token: string): Record<string, unknown> {
  const payload = token.split('.')[1];
  const decoded = JSON.parse(atob(payload.replace(/-/g, '+').replace(/_/g, '/')));
  console.table(decoded); // roles, groups, aud, scope, permissions – eyeball the diff
  return decoded;
}
```

```
// Client-side defensive fix, once the actual claim name is confirmed (e.g., IdP sends
// `groups` but the app's authGuard/role-check was hardcoded to look for `roles`):
export function extractRoles(claims: Record<string, unknown>): string[] {
  // Support multiple possible claim shapes across identity providers instead of
  // assuming one hardcoded shape – SSO integrations often need this flexibility.
  return (claims['roles'] as string[])
    ?? (claims['groups'] as string[])
    ?? (claims['https://yourapp.com/roles'] as string[])
    ?? [];
}
```

The client-side Angular fix is usually secondary here (the real fix is almost always backend claims-mapping/audience config), but the Angular app's role-checking logic (guards, `*ngIf`-equivalent structural directives gating UI by role) should be defensively written to handle multiple possible claim shapes and to fail loudly/log clearly rather than silently defaulting to "no roles" with no diagnostic trail, which is exactly what made this bug hard to triage in the first place.

Interviewer intent: Tests systematic debugging methodology (isolate authn vs authz, compare claim shapes, check `aud`) rather than guessing, and awareness that SSO integrations frequently break at the claims-mapping layer, not the cryptographic/token-validation layer.

Scenario 17: Multiple SSO providers (Google, Microsoft, Okta) — the callback route needs to know which one

Situation: The app added support for logging in via Google, Microsoft Entra ID, and Okta (three separate OIDC providers, three separate client IDs/issuer URLs) in addition to the original password login. All three redirect back to the same `/auth/callback` route. The team's current implementation hardcodes Google's token endpoint in the callback handler, so Microsoft and Okta logins silently fail at the final code-exchange step.

Question: How do you architect a callback route that correctly handles multiple OIDC providers without hardcoding provider-specific logic in one place?

Answer: The `state` parameter (already carrying the return URL per Scenario 9) is the natural place to also carry which provider initiated the flow, since it's the one piece of data guaranteed to round-trip unmodified through the entire redirect. Design a small provider-registry/strategy pattern so the callback route looks up the right config by a `provider` key rather than hardcoding one provider's specifics.

```
// oidc-providers.config.ts
export interface OidcProviderConfig {
  id: 'google' | 'microsoft' | 'okta';
  clientId: string;
  authorizeUrl: string;
  tokenUrl: string;
  redirectUri: string;
  scope: string;
}

export const OIDC_PROVIDERS: Record<string, OidcProviderConfig> = {
  google: {
    id: 'google',
```

```

    clientId: 'GOOGLE_CLIENT_ID',
    authorizeUrl: 'https://accounts.google.com/o/oauth2/v2/auth',
    tokenUrl: 'https://oauth2.googleapis.com/token',
    redirectUri: 'https://app.example.com/auth/callback',
    scope: 'openid email profile',
  },
  microsoft: {
    id: 'microsoft',
    clientId: 'MS_CLIENT_ID',
    authorizeUrl: 'https://login.microsoftonline.com/common/oauth2/v2.0/authorize',
    tokenUrl: 'https://login.microsoftonline.com/common/oauth2/v2.0/token',
    redirectUri: 'https://app.example.com/auth/callback',
    scope: 'openid email profile',
  },
  okta: {
    id: 'okta',
    clientId: 'OKTA_CLIENT_ID',
    authorizeUrl: 'https://your-org.okta.com/oauth2/default/v1/authorize',
    tokenUrl: 'https://your-org.okta.com/oauth2/default/v1/token',
    redirectUri: 'https://app.example.com/auth/callback',
    scope: 'openid email profile',
  },
};

```

```

// sso.service.ts
@Injectable({ providedIn: 'root' })
export class SsoService {
  private http = inject(HttpClient);

  redirectToProvider(providerId: keyof typeof OIDC_PROVIDERS, returnUrl: string) {
    const config = OIDC_PROVIDERS[providerId];
    const stateNonce = crypto.randomUUID();
    const state = btoa(JSON.stringify({ provider: providerId, returnUrl, nonce: stateNonce }));
    sessionStorage.setItem('oidc_state_nonce', stateNonce);

    const url = new URL(config.authorizeUrl);
    url.searchParams.set('client_id', config.clientId);
    url.searchParams.set('redirect_uri', config.redirectUri);
    url.searchParams.set('response_type', 'code');
    url.searchParams.set('scope', config.scope);
    url.searchParams.set('state', state);
    window.location.href = url.toString();
  }

  exchangeCode(providerId: keyof typeof OIDC_PROVIDERS, code: string) {
    const config = OIDC_PROVIDERS[providerId];
    // Note: in production this exchange should happen server-side (never expose
    // a client_secret in Angular code) – the app calls YOUR backend, which
    // knows which provider's token endpoint to hit using the same provider key.
    return this.http.post('/auth/exchange', { provider: providerId, code });
  }
}

```

```
}
```

```
// callback.component.ts
export class CallbackComponent {
  private sso = inject(SsoService);
  private route = inject(ActivatedRoute);
  private router = inject(Router);

  constructor() {
    const returnedState = this.route.snapshot.queryParamMap.get('state');
    const code = this.route.snapshot.queryParamMap.get('code');
    const storedNonce = sessionStorage.getItem('oidc_state_nonce');
    if (!returnedState) { this.router.navigateByUrl('/login'); return; }

    const { provider, returnUrl, nonce } = JSON.parse(atob(returnedState));
    if (nonce !== storedNonce) { this.router.navigateByUrl('/login?error=state_mismatch');
    return; }

    this.sso.exchangeCode(provider, code!).subscribe(() => {
      this.router.navigateByUrl(returnUrl || '/dashboard');
    });
  }
}
```

Critically, the actual authorization-code-for-token exchange (`tokenUrl` call) should happen on your backend, never directly from Angular, because that exchange typically requires a `client_secret` for confidential clients — something that must never ship in frontend code. The Angular app's job is only to redirect to the right `authorizeUrl` and, on return, tell your own backend "here's the code and which provider it came from," letting the backend hold the actual provider-specific secrets and token-endpoint logic. This also naturally solves the original bug: the callback route is now provider-agnostic, driven entirely by the `provider` field carried through `state`.

Interviewer intent: Tests whether the candidate can generalize a single-provider OIDC integration into a multi-provider strategy pattern, and whether they know client secrets must never live in Angular/frontend code even under time pressure to "just make Microsoft login work like Google's."

Scenario 18: Role changes don't take effect until the user manually logs out and back in

Situation: An admin promotes a user from `viewer` to `editor` in the admin panel. The user, already logged in with an active session, still sees `viewer`-only UI and gets 403s on editor actions for up to 15 minutes (their access token lifetime) — sometimes users complain it takes even longer because they don't hit any 401 until their next natural token expiry.

Question: How do you make role/permission changes propagate to already-logged-in users faster, without abandoning the performance benefits of stateless JWTs (i.e., without checking the DB on every single request)?

Answer: This is an inherent tradeoff of putting authorization claims *inside* a signed, stateless JWT: the token is a snapshot of permissions at issuance time, and by design the API doesn't hit the database to re-verify roles on every request (that's the whole performance point of JWTs). Fully "instant" propagation would require abandoning statelessness for authorization checks specifically. Practical mitigations, layered by how urgently propagation is needed:

1. **Shorten access token lifetime for authorization-sensitive apps** (e.g., 5 minutes instead of 15-60) so the worst-case staleness window shrinks — cheap, but doesn't solve it, just narrows the window, and adds refresh-call overhead.
2. **Push-based invalidation for role changes specifically:** when an admin changes a role, push a targeted signal (via WebSocket/SSE, or a lightweight polling check) to that specific user's active sessions telling the client to force-refresh its token immediately, bypassing the normal proactive-refresh schedule.
3. **Server-side permission check on sensitive actions, independent of JWT claims:** for genuinely high-stakes operations (e.g., deleting a production resource), don't trust the JWT's embedded role claim at all — have the API re-check current permissions against the database for that specific call, accepting the performance cost only where it matters, while leaving low-stakes reads to trust the JWT claim as-is.

```
// role-change-listener.service.ts – real-time forced token refresh on role change
import { Injectable, inject } from '@angular/core';
import { AuthRefreshService } from '../auth-refresh.service';

@Injectable({ providedIn: 'root' })
export class RoleChangeListenerService {
  private auth = inject(AuthRefreshService);
  private socket?: WebSocket;

  connect(userId: string) {
    this.socket = new WebSocket(`wss://api.example.com/ws/role-updates?userId=${userId}`);
    this.socket.onmessage = (event) => {
      const msg = JSON.parse(event.data);
      if (msg.type === 'ROLE_CHANGED') {
        // Force an immediate refresh so the NEXT access token reflects the new role,
        // instead of waiting up to the full remaining TTL of the current one.
        this.auth.refreshAccessToken().subscribe();
      }
    };
  }
}
```

```
// For the "sensitive action" mitigation – server pseudocode context the interviewer
// wants you to be aware of, even though it's not Angular code:
// DELETE /production-resources/:id
// -> middleware does NOT just check req.jwt.role === 'admin'
// -> instead: const freshRole = await db.getUserRole(req.jwt.sub); check freshRole
```

The honest framing to give in an interview: there is no purely-client-side fix here, and no way to get instant propagation while keeping pure stateless JWT authorization for every request — the real answer is choosing, per endpoint, how much staleness risk is acceptable, and using push notifications to minimize (not eliminate) the average staleness window for the common case, while falling back to authoritative DB checks for the

genuinely dangerous actions.

Interviewer intent: Tests whether the candidate understands the fundamental stateless-JWT-authorization tradeoff (performance vs. staleness) and can propose a layered, endpoint-by-endpoint mitigation strategy rather than either naively promising "instant" propagation or shrugging it off as unsolvable.

Scenario 19: A `HttpInterceptorFn` needs to skip adding the Authorization header for a specific third-party API call

Situation: The app's `authInterceptor` is global (registered once via `provideHttpClient(withInterceptors([authInterceptor]))`) and blindly attaches the app's own JWT to every outgoing `HttpClient` request. A new feature calls a third-party public API (e.g., a weather API, a maps geocoding endpoint) directly from Angular using the same injected `HttpClient` — and the interceptor is now leaking the app's internal auth `Bearer` token to a completely unrelated third-party domain.

Question: How do you scope an interceptor so it only attaches credentials to your own API, never to arbitrary outbound requests?

Answer: This is a real and surprisingly common security leak: a global interceptor with no origin check will happily attach your app's bearer token as an `Authorization` header on a request to `api.openweathermap.org` if that call goes through the same `HttpClient`. The fix is to explicitly allowlist which request URLs should receive the token, matched against your own API's base URL, and pass through everything else untouched.

```
// auth.interceptor.ts – scoped to only your own API origin(s)
import { HttpInterceptorFn } from '@angular/common/http';
import { inject } from '@angular/core';
import { environment } from '../environments/environment';

const API_ALLOWLIST = [environment.apiUrl]; // e.g., 'https://api.yourapp.com'

function isOwnApiRequest(url: string): boolean {
  return API_ALLOWLIST.some((base) => url.startsWith(base));
}

export const authInterceptor: HttpInterceptorFn = (req, next) => {
  if (!isOwnApiRequest(req.url)) {
    return next(req); // third-party call – pass through untouched, no token attached
  }

  const auth = inject(AuthRefreshService);
  const token = auth.getAccessToken();
  const authedReq = token
    ? req.clone({ setHeaders: { Authorization: `Bearer ${token}` } })
    : req;

  return next(authedReq);
};
```

An alternative, arguably even safer pattern for third-party calls: use a separate, dedicated `HttpClient` instance (via `HttpBackend` directly, bypassing the interceptor chain entirely) for any outbound call to a non-first-party API, so there's no reliance on remembering to allowlist correctly — the third-party traffic architecturally never passes through your auth interceptor at all.

```
// third-party-http.service.ts
import { Injectable, inject } from '@angular/core';
import { HttpBackend, HttpClient } from '@angular/common/http';

@Injectable({ providedIn: 'root' })
export class ThirdPartyHttpService {
  // HttpClient built directly from HttpBackend skips ALL registered interceptors –
  // a clean escape hatch for calls that must never see your app's auth logic.
  private rawHttp = new HttpClient(inject(HttpBackend));

  getWeather(city: string) {
    return this.rawHttp.get(`https://api.weather-provider.com/v1/current?city=${city}`);
  }
}
```

Both approaches are valid; the allowlist approach keeps a single `HttpClient` for simplicity, while the `HttpBackend` escape hatch is more foolproof against future developers forgetting to update the allowlist. Either way, the underlying lesson is: **never assume a global interceptor is safe by default** — always reason explicitly about which origins should receive your app's credentials, since attaching an internal bearer token to a third-party request is a genuine credential-leak vulnerability (the third party's server, or anyone logging its access logs, now has a valid token for your app).

Interviewer intent: Tests awareness that global interceptors are a blunt instrument and can leak credentials to unrelated APIs if not scoped carefully — a security-adjacent scenario that many candidates who've only worked with a single first-party API have never had to think about.

Scenario 20: A user reports being logged out immediately after changing their password on another device

Situation: A user changes their password on their laptop. Moments later, their phone app (still logged in from before the password change) throws a 401 and force-logs-out. The user is confused ("I didn't touch my phone, why did it log me out?") and files it as a bug, but the security team says this is actually working as intended and closes it as "not a bug."

Question: Explain why this is correct security behavior, and how you'd design the client UX so it doesn't feel like a confusing bug to the user.

Answer: This is correct and important behavior: changing a password should invalidate *all* existing sessions/refresh tokens issued before the change, precisely to protect against the scenario where an attacker has a stolen session (e.g., a leaked refresh token) — if password changes didn't revoke prior sessions, changing your password in response to a suspected compromise would do nothing to kick out the attacker's already-authenticated session. The server should maintain something like a `tokenValidAfter` timestamp on the user record, and reject any refresh token/access token issued before that timestamp, regardless of its own `exp`.

The Angular-side responsibility isn't to prevent this (you shouldn't try to prevent it — that would be a security regression) but to make the *resulting UX* clear rather than confusing: distinguish "your session ended because of a security-relevant event" from a generic/ambiguous 401, so the user understands why, instead of feeling like the app randomly broke.

```
// auth.interceptor.ts – surface a specific, user-legible reason for forced logout
export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const auth = inject(AuthRefreshService);
  const notifications = inject(NotificationService);
  const token = auth.getAccessToken();
  const authedReq = token ? req.clone({ setHeaders: { Authorization: `Bearer ${token}` } })
    : req;

  return next(authedReq).pipe(
    catchError((err: unknown) => {
      if (err instanceof HttpResponse && err.status === 401) {
        const reason = err.error?.reason; // server includes a machine-readable reason code
        return auth.refreshAccessToken().pipe(
          switchMap((freshToken) => next(req.clone({ setHeaders: { Authorization: `Bearer ${freshToken}` } }))),
          catchError((refreshErr: HttpResponse) => {
            const refreshReason = refreshErr.error?.reason;
            if (refreshReason === 'password_changed') {
              notifications.show('Your password was changed recently. Please log in again for your security.');
```

This requires the backend to include a structured, machine-readable reason code (`password_changed`, `session_revoked`, `token_expired`, `suspicious_activity`) in its 401/refresh-failure response body, not just a generic "unauthorized" — the Angular app can then translate that into an honest, specific, reassuring message rather than a bare "session expired," which turns a security feature working exactly as intended into a moment of user trust ("oh good, the app is actually protecting me") rather than a moment of confusion/support-ticket generation ("the app randomly logged me out, is it broken?").

Interviewer intent: Tests whether the candidate can distinguish "this is a bug" from "this is a security feature with a UX communication gap," and whether they think about auth from a holistic product-security perspective, not just token mechanics — a good signal for seniority.

Quick Revision Cheat Sheet

- **Single-flight refresh:** Use `shareReplay(1)` around one shared refresh `observable` so concurrent 401s piggyback on one HTTP call instead of firing N parallel refreshes (Scenarios 1, 6).
- **Proactive + reactive refresh:** Schedule a refresh before `exp` (proactive) in addition to handling 401s (reactive) — purely reactive refresh loses in-flight form data on long sessions (Scenario 2).
- **Idempotency over blind retries:** Never blindly retry non-idempotent POST/PATCH after a refresh unless the 401 was a genuine auth rejection; use server-side idempotency keys for true safety (Scenario 3).
- **Cross-tab identity sync:** `localStorage` is shared across tabs with no built-in invalidation signal — use `BroadcastChannel / storage` events and hard-reload on identity mismatch rather than patching in-memory state (Scenarios 4, 5).
- **Cross-tab refresh locking:** `shareReplay` only de-duplicates within one tab; use the Web Locks API (`navigator.locks`) to prevent refresh-token-rotation reuse-detection false positives across tabs (Scenario 6).
- **JWT ≠ session cookie semantics:** Sliding expiration, instant revocation, and server-side logout all require deliberate re-engineering when migrating from stateful sessions to stateless JWTs — nothing is "free" (Scenario 7).
- **CSRF vs XSS tradeoff:** Cookies trade XSS-safety for CSRF-risk; `localStorage` JWTs invert that. The BFF pattern (refresh token in `HttpOnly` cookie, access token in memory only) gets the best of both (Scenario 8).
- **OIDC state param:** Carries both the CSRF defense for the auth flow AND the "return-to" URL through the redirect round-trip — always validate it on return, and always validate `returnurl` is same-origin to avoid open redirects (Scenario 9).
- **Gate bootstrap on silent SSO:** Use `provideAppInitializer` to block the router's first navigation decision until a silent-SSO/session-restore check resolves, avoiding a flash of the login page (Scenario 10).
- **Auth state singletons:** Auth-holding services must be `providedIn: 'root'` and never re-declared in route/component `providers` arrays, or DI silently creates divergent instances with disconnected state (Scenario 11).
- **JWT expiry is a clock-trust problem:** Never gate request-sending purely on a locally-decoded `exp`; let the server's 401 be authoritative, and apply server-side `clockTolerance` /leeway for clock skew (Scenario 12).
- **Three-state auth model:** Model auth state as `unknown / authenticated / unauthenticated`, not a boolean defaulting to `false` — otherwise "not yet checked" is indistinguishable from "confirmed logged out," causing guard race conditions on hard refresh (Scenario 13).
- **Consume emitted values, not stale closures:** After an async refresh inside a `switchMap`, always use the token *emitted* by the refresh call, never a `token` variable captured earlier in the same function (Scenario 14).
- **Safari ITP purges localStorage:** After 7 days of no direct top-level visits, Safari wipes script-writable storage — store refresh tokens in server-set `HttpOnly` cookies, which are exempt, to avoid mysterious weekly logouts (Scenario 15).

- **SSO bugs are usually claims-mapping, not crypto:** When SSO users authenticate fine but get 403 everywhere, check claim shape (`roles` vs `groups` vs namespaced claims) and `aud` mismatches before suspecting token validation itself (Scenario 16).
- **Never hardcode one OIDC provider:** Use a provider registry keyed by an ID carried through the `state` param, and always exchange the auth code for tokens via your own backend — never expose `client_secret` in Angular (Scenario 17).
- **Stateless JWTs mean stale authorization:** Role/permission changes won't propagate faster than the access token's TTL unless you add push-based forced-refresh or fall back to authoritative DB checks for high-stakes actions (Scenario 18).
- **Scope interceptors to your own origin:** A global `HttpInterceptorFn` will leak your app's bearer token to third-party APIs unless explicitly allowlisted by URL, or unless third-party calls use a raw `HttpClient` built from `HttpBackend` to bypass interceptors entirely (Scenario 19).
- **Some "random" logouts are working as intended:** Password changes should invalidate all prior sessions — the fix is UX communication (a specific reason code and message), not preventing the revocation (Scenario 20).
- **Guards should return observables/promises, not just booleans,** so the router can genuinely wait on async auth resolution instead of racing ahead of it (Scenarios 9, 13).

Created By - Durgesh Singh